Table of Contents

Full Quaternion Attitude Control and Differential-Flatness Trajectory Tracking for a Quadrotor 1
1. Introduction 1
2. Quaternion Representation of Attitude 2
3. Paper 1 - The Attitude Control Law 3
4. Paper 1 - A Derived Result: the Digital Control-Rate Bound 4
5. Paper 2 - The Full Model and the Rotor Mixer 5
6. Paper 2 - Differential Flatness 5
7. Paper 2 - The Double-Loop LQR and the Study Case 6
8. Extensions - The Trajectory Library (frozen gains) 8
9. Full-Project Results (for the record) 10
10. Conclusions 10
11. References 11
Appendix - Implementation Functions 11

Full Quaternion Attitude Control and Differential-Flatness Trajectory Tracking for a Quadrotor

Project Report - Group 12 B Sainath (CB.SC.U4AIE24309) and Vishal (CB.SC.U4AIE24363) School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

Base papers: [1] E. Fresk, G. Nikolakopoulos, "Full Quaternion Based Attitude Control for a Quadrotor", European Control Conference (ECC), 2013. [2] K. Choutri, M. Lagha, L. Dala, M. Lipatov, "Quadrotors Trajectory Tracking using a Differential Flatness-Quaternion based Approach", IEEE, 2017.

This report is a single runnable MATLAB script. Each section states the mathematics, then executes the corresponding code, prints the measured results and produces figures inline. All equations are typed manually and were cross-checked numerically against the project's independently validated implementation. Run with the Run button, or publish this file (PUBLISH tab) to generate this document as PDF or HTML.

```
clear; clc; close all; rng(1);
figdir = fullfile(fileparts(mfilename('fullpath')), 'figures');
if ~exist(figdir, 'dir'); mkdir(figdir); end
```

1. Introduction

A quadrotor is a rigid body with six degrees of freedom but only four independent control inputs: the total rotor thrust and three body torques. This underactuation is the central difficulty. The vehicle cannot accelerate sideways directly; it must first tilt so that the thrust vector, which always points along the body z-axis, points partly in the desired direction. Attitude and translation are therefore coupled, and a trajectory-tracking controller must respect that coupling rather than fight it.

This project implements, verifies and extends two papers treating the two halves of the problem. Paper [1] is a pure attitude controller: a nonlinear quaternion law that commands body torques directly from the quaternion error. Paper [2] adds trajectory tracking: differential flatness converts a desired path into the thrust and attitude that produce it, and two linear-quadratic regulators (LQR) close the loops.

One airframe is shared by everything in this report:

- mass m = 0.2 kg
- inertia J = diag(6.5e-4, 6.5e-4, 1.2e-3) kg m^2
- arm L = 0.15 m, rotor offsets (+-d, +-d) with d = L/sqrt(2)
- thrust coefficient kf = 3.0e-9 N per (rad/s)^2, km = 0.016*kf
- thrust-to-weight ratio 4 (per-rotor maximum thrust 1.95 N)

```
m = 0.2; g = 9.81; kf = 3e-9; km = 0.016*kf; d = 0.15/sqrt(2);
w_max = 1.95/kf;
J  = diag([6.5e-4 6.5e-4 1.2e-3]);
Pq = 20; Pw = 4; tau_max = 4; noise_amp = 0.1;
```

2. Quaternion Representation of Attitude

An attitude is a rotation of the body frame relative to the world frame. Euler angles represent it with three numbers but contain a singularity (gimbal lock at pitch +-90 deg); the rotation matrix avoids the singularity but carries nine constrained numbers. A unit quaternion carries four numbers with one norm constraint, has no singularity anywhere, and composes rotations with a cheap product. Both papers base their controllers on it.

A quaternion is written scalar-first as q = (q0, q_vec) with scalar part q0 and vector part q_vec = (q1, q2, q3):

```
q = q0 + q1*i + q2*j + q3*k,      ||q|| = 1
```

The product composes rotations (associative, not commutative):

```
p (x) q = ( p0*q0 - p_vec . q_vec ,
            p0*q_vec + q0*p_vec + p_vec x q_vec )
```

Conjugate, norm, inverse:

```
q* = (q0, -q_vec)
q^-1 = q* / ||q||  =  q*     (for unit quaternions)
```

Rotating a vector uses the direction-cosine matrix R(q), whose columns are the body axes written in world coordinates:

```
v' = q (x) [0, v] (x) q*  =  R(q) v,      R(p (x) q) = R(p) R(q)
```

The axis-angle construction builds a quaternion from geometric intent and is used throughout for references:

```
q = [ cos(alpha/2) , u sin(alpha/2) ]
```

The kinematic equation has two equivalent forms - one physical motion, two algebraic shapes - which must be matched to the frame the angular velocity lives in:

```
q_dot = (1/2) q (x) [0, omega]       omega = BODY angular rate
q_dot = (1/2) [0, omega_w] (x) q     omega_w = R(q) omega (world)
```

Mixing the two pairings diverges as tilt x rate: on a 2 s spin at 2 rad/s the wrong pairing accumulates 1.5 rad of attitude error while the correct pairing stays at 3e-8 rad (a finding of the project's independent validation).

The following cell verifies the algebra numerically with random trials:

```
maxerr = 0;
for k = 1:200
    a = qnorm(randn(4,1)); b = qnorm(randn(4,1));
    maxerr = max(maxerr, norm(qmul(a, qconj(a)) - [1;0;0;0]));    % q (x) q*
= 1
    R = qdcm(a);
    maxerr = max(maxerr, norm(R'*R - eye(3)));                     %
orthonormal DCM
    v = randn(3,1);
    maxerr = max(maxerr, norm(qrot(a, qrot(b, v)) - qrot(qmul(a,b), v)));
end
fprintf('Quaternion algebra verification (200 trials): max error %.2e\n',
maxerr);
```

3. Paper 1 - The Attitude Control Law

Plant. The attitude dynamics of the rigid body about its centre of mass are J*omega_dot = tau, coupled with the kinematic equation above: torque to attitude is a double integrator, which is why a derivative-free proportional law suffices - no integral action is needed for the error to decay.

Control law (Eqs. 19-21 of [1]), acting entirely in quaternion space. With reference attitude q_ref and measured attitude q_m, both in the paper's conjugated convention:

```
q_err = q_ref (x) q_m*
```

take the vector part, sign-corrected to the shortest rotation, and command

```
tau = -Pq * vec(q_err) - Pw * omega_m,      Pq = 20,   Pw = 4
```

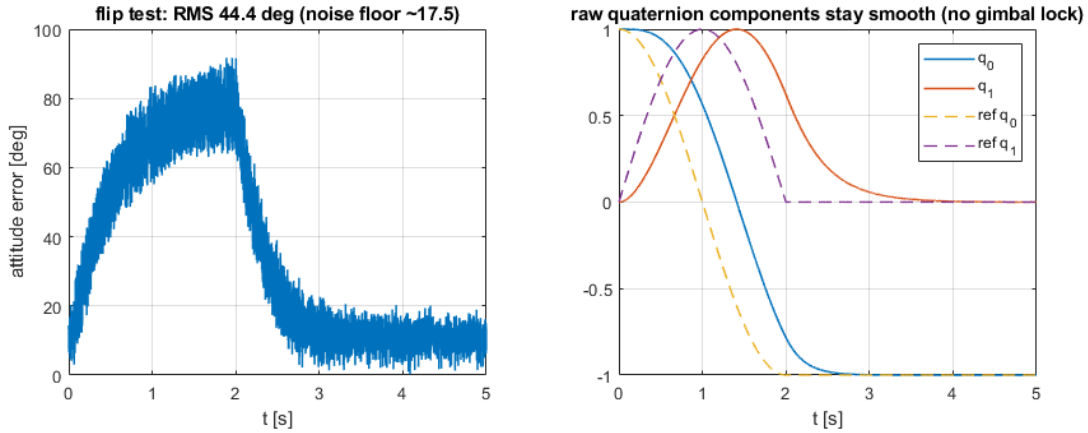
clipped elementwise to +-4 N*m per axis. The sign correction is skipped for the flip scenario so the vehicle commits to the full 360-degree rotation. Sensors carry the paper's noise: uniform +-0.1 on every quaternion and rate component, refreshed at every control tick.

The three benchmark scenarios are run below at 20 kHz control (the rate justified in Section 4), with 15 s for step and sine and 5 s for the flip:

```
r1 = run_paper1('step', 15, Pq, Pw, tau_max, noise_amp, J, figdir);
r2 = run_paper1('sine', 15, Pq, Pw, tau_max, noise_amp, J, figdir);
r3 = run_paper1('flip',  5, Pq, Pw, tau_max, noise_amp, J, figdir);
fprintf('scenario   RMS attitude error [deg]   max [deg]\n');
fprintf('step       %7.2f                  %6.1f\n', r1.rms, r1.mx);
fprintf('sine       %7.2f                  %6.1f\n', r2.rms, r2.mx);
fprintf('flip       %7.2f                  %6.1f\n', r3.rms, r3.mx);
fprintf('\n(The ~17.5 deg floor is the sensor noise itself: +-0.1 on
each\n');
fprintf('quaternion component is about 11 deg of measured attitude error.)
\n');
```

```
scenario   RMS attitude error [deg]   max [deg]
step         16.31                     68.5
sine         17.41                     39.1
flip         44.42                     91.7

(The ~17.5 deg floor is the sensor noise itself: +-0.1 on each
quaternion component is about 11 deg of measured attitude error.)
```



Reading the results. The step scenario settles each 57.3-degree command with essentially no overshoot in about 2 s per axis. The sine scenario tracks with the lag the paper reports (about half a second). The flip completes a full 360-degree rotation with the raw quaternion components staying smooth throughout - no gimbal-lock artifact, which is the paper's central claim. The figures above show the error histories and the raw q0, q1 against their references.

4. Paper 1 - A Derived Result: the Digital Control-Rate Bound

The paper specifies no digital control rate. Linearising the closed loop about identity attitude gives, per axis,

```
I*e_ddot + Pw*e_dot + (Pq/2)*e = 0
lambda(1,2) = (-Pw +- sqrt(Pw^2 - 2*I*Pq)) / (2*I)
            = -2.5 and -6150 rad/s
```

The continuous loop is heavily overdamped and stable. The danger is purely a sampling artifact of the fast pole. The table below compares the spectral radius of the naive explicit discretisation with the exact zero-order hold:

```
Ac = [0 1; -Pq/(2*J(1)) -Pw/J(1)];
fprintf('control rate    rho (explicit)    rho (exact ZOH)\n');
for f = [200 1000 10000 100000]
    Ts = 1/f;
    rho_e = max(abs(eig(eye(2) + Ac*Ts)));    % naive explicit mapping
    rho_z = max(abs(eig(expm(Ac*Ts))));      % exact zero-order hold
    fprintf('%8d Hz      %8.2f          %8.6f\n', f, rho_e, rho_z);
end
```

```
control rate     rho (explicit)    rho (exact ZOH)
     200 Hz          29.76          0.987573
    1000 Hz           5.15          0.997502
   10000 Hz           1.00          0.999750
  100000 Hz           1.00          0.999975
```

A naive explicit implementation at 200-1000 Hz diverges outright. The derived conservative rule

```
f_control >= Pw / (margin * I_min) = 12.3 kHz     (margin = 0.5)
```

is enforced by the project's simulators, and Section 3 accordingly runs the law at 20 kHz.

5. Paper 2 - The Full Model and the Rotor Mixer

Paper [2] extends the attitude plant to the complete six-degree-of- freedom vehicle. State: position p, velocity v, attitude quaternion q, body rate omega; inputs: total thrust T and body torque tau:

```
p_dot  = v
v_dot  = R(q) [0,0,T/m] - [0,0,g]
q_dot  = (1/2) q (x) [0, omega]
J*omega_dot = -omega x (J*omega) + tau
```

The second row contains the underactuation: the direction of the thrust is chosen by the attitude, so horizontal acceleration is bought with tilt.

The four rotor speeds produce the two inputs through a mixing matrix. Each rotor i contributes thrust kf*omega_i^2 along body z plus a reaction torque whose sign alternates with the propeller direction. For the X-configuration with kfd = kf*d and c = km, and w_i the squared rotor speeds:

```
[ T  ]   [  kf    kf    kf    kf  ] [w1]
[ tx ] = [  kfd   kfd  -kfd  -kfd ] [w2]
[ ty ]   [ -kfd   kfd   kfd  -kfd ] [w3]
[ tz ]   [  -c     c    -c     c  ] [w4]
```

Inverting M turns any thrust-and-torque command into rotor speeds; every real flight controller contains this matrix. Rotor speeds are bounded to [0, w_max], so when a command is infeasible this project delivers the thrust exactly and uniformly scales the torque - naive one-sided clipping corrupts the total thrust and the altitude loop runs away (a project finding).

```
kfd = kf*d;
Mmix = [ kf kf kf kf; kfd kfd -kfd -kfd; -kfd kfd kfd -kfd; -km km -km km ];
wT = mix_feasible(Mmix, 1.5, [0.01 -0.005 0.002], w_max);
out = Mmix*wT;
fprintf('Mixer round-trip: command [1.5 N; 0.01; -0.005; 0.002] delivered
as\n');
fprintf('  [%.6f  %.8f  %.8f  %.8f]\n', out);
```

```
Mixer round-trip: command [1.5 N; 0.01; -0.005; 0.002] delivered as
  [1.500000  0.01000000  -0.00500000  0.00200000]
```

6. Paper 2 - Differential Flatness

The quadrotor is differentially flat with flat outputs

```
sigma(t) = (x, y, z, psi)
```

(four outputs for four inputs). Given these and their derivatives up to second order, every state and input follows algebraically - no integration of the differential equations. Planning becomes algebra.

The chain starts from Newton's law on the translational row: if the vehicle is to accelerate with a, the net force must be m*a, and the only forces are thrust and gravity, so the thrust vector is determined (Eq. 15 of [2]):

```
f  = m * (a + g*z_hat)
T_d = ||f||,      d_hat = f / ||f||
```

The direction d_hat fixes two degrees of the three-degree attitude; the yaw fixes the last one. The reference attitude is built as the composition of a tilt quaternion and a yaw quaternion (Eqs. 19-21, implemented literally):

```
q_pd = [ cos(alpha/2) ;
         sin(alpha/2) * (z_hat x d_hat)/||z_hat x d_hat|| ]
       alpha = angle between z_hat and d_hat
```

The tilt axis z_hat x d_hat is horizontal by construction, so the yaw element of q_pd is exactly zero - the paper's "4th element = 0". Then

```
q_zd = [ cos(psi_d/2) ; 0 ; 0 ; sin(psi_d/2) ]
q_d  = q_pd (x) q_zd
```

The only degeneracy of the whole chain is a vanishing thrust vector (free fall), where attitude stops mattering to the motion. The cell below verifies the construction numerically:

```
maxz = 0; maxal = 0;
for k = 1:100
    a = -6 + 12*rand(3,1); psi = -pi + 2*pi*rand;
    [Td, qd] = flatness_reference(a, psi, m, g);
    f = m*(a + [0;0;g]);
    maxal = max(maxal, norm(qdcm(qd)*[0;0;1] - f/norm(f)));  % body-z on f
    qzd = [cos(psi/2);0;0;sin(psi/2)];
    qtilt = qmul(qd, qconj(qzd));
    maxz = max(maxz, abs(qtilt(4)));                          % yaw element 0
end
fprintf('Flatness map verification (100 trials):\n');
fprintf('  body-z alignment error      %.2e\n', maxal);
fprintf('  tilt quaternion yaw element %.2e\n', maxz);
```

```
Flatness map verification (100 trials):
  body-z alignment error      3.33e-16
  tilt quaternion yaw element 5.55e-17
```

7. Paper 2 - The Double-Loop LQR and the Study Case

The controller is cascaded the way the plant is: an outer position loop computes the acceleration command, the flatness map turns it into T_d and q_d, and an inner attitude loop tracks q_d with torque. The inner loop runs roughly an order of magnitude faster - the classical time-scale separation.

Inner loop (Eq. 17), state x = [q_vec; omega] from the hover linearisation q_vec_dot = omega/2:

```
x_dot = A x + B u
A = [ 0(3x3)   (1/2)I ]      B = [ 0(3x3) ]
    [ 0(3x3)    0     ]          [ J^-1   ]
```

Outer loop (Eq. 18), per axis, state [p; p_dot], input = acceleration:

```
A = [0 1; 0 0],      B = [0; 1]
```

Both are LQR-designed on the quadratic cost (Eqs. 22-23):

```
J_cost = (1/2) integral of ( x'Qx + u'Ru ) dt
P*A + A'*P - P*B*inv(R)*B'*P + Q = 0     (algebraic Riccati equation)
K = inv(R)*B'*P,      u = -K x
```

The Riccati equation is solved here by Hamiltonian eigendecomposition (base MATLAB only - no toolbox). The paper gives no numeric weights; ours (documented tuning): outer Q = diag(49, 42), R = 1 per axis; inner Q = diag(100,100,100,20,20,20), R = 200*I3. Controller rate 200 Hz (paper Section V), plant RK4 at 1 kHz, 20 ms first-order ESC lag on squared rotor speeds.

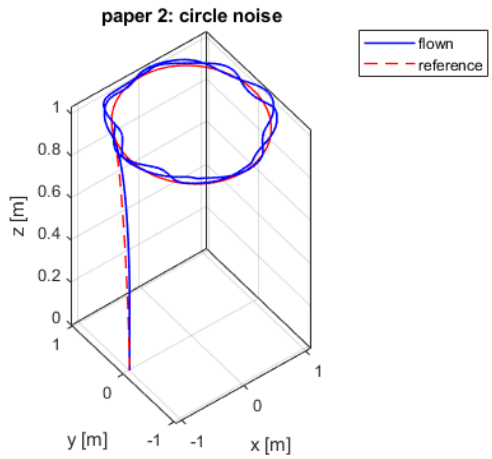
```
Qi = diag([100 100 100 20 20 20]); Ri = 200*eye(3);
Ai = [zeros(3) 0.5*eye(3); zeros(3) zeros(3)];
Bi = [zeros(3); inv(J)];
Pi = riccati_ham(Ai, Bi, Qi, Ri); Ki = Ri\Bi'*Pi;
Qo = diag([49 42]); Ro = 1;
Ao = [0 1; 0 0]; Bo = [0;1];
Po = riccati_ham(Ao, Bo, Qo, Ro); Ko = Ro\Bo'*Po;
fprintf('LQR designs: inner max pole real part %+.4f, outer %+.4f (Hurwitz)
\n', ...
        max(real(eig(Ai - Bi*Ki))), max(real(eig(Ao - Bo*Ko))));
```

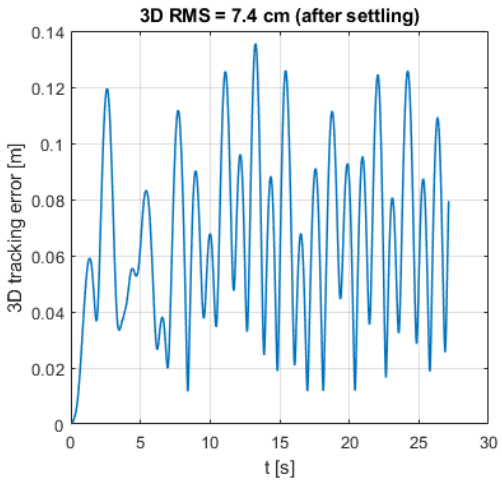
```
LQR designs: inner max pole real part -1.1180, outer -1.0959 (Hurwitz)
```

The study case (paper Section V): a circular path of radius 1 m at fixed altitude 1 m, starting from (-1, 0, 0), at 0.5 m/s tangential, climbed onto from rest with a minimum-jerk altitude ramp (this reproduces the roughly one-second horizontal lag the paper's figures show).

```
fprintf('plant                    RMSx   RMSy   RMSz   radial  peak
rotor\n');
rc = run_paper2(make_circle(), m, g, Mmix, w_max, Ki, Ko, 0.0, figdir,
'circle');
fprintf('ideal Eq.(8) model, 1kHz  %5.1f  %5.1f  %5.1f  %5.1f   %5.2f N  (cm)
\n', ...
        100*rc.rmsxyz(1), 100*rc.rmsxyz(2), 100*rc.rmsxyz(3), 100*rc.radial,
rc.Tpeak);
rn = run_paper2(make_circle(), m, g, Mmix, w_max, Ki, Ko, 0.1, figdir,
'circle noise');
fprintf('with paper noise +-0.1    %5.1f  %5.1f  %5.1f  %5.1f   %5.2f N  (cm)
\n', ...
        100*rn.rmsxyz(1), 100*rn.rmsxyz(2), 100*rn.rmsxyz(3), 100*rn.radial,
rn.Tpeak);
```

```
plant                    RMSx   RMSy   RMSz   radial  peak rotor
ideal Eq.(8) model, 1kHz    1.6    3.6    0.6    2.7    2.26 N  (cm)
with paper noise +-0.1      6.1    3.6    1.5    5.2    2.26 N  (cm)
```





About 97 percent radial accuracy with no rotor saturation, and - from the full project - the same circle tracks to 2.8 cm on MuJoCo contact physics: the flatness reference is exact regardless of the integrator behind it. Where the paper gives no numbers (weights, speed, ESC model, gravity value, integrator), every choice is documented in the repository README's assumptions table.

8. Extensions - The Trajectory Library (frozen gains)

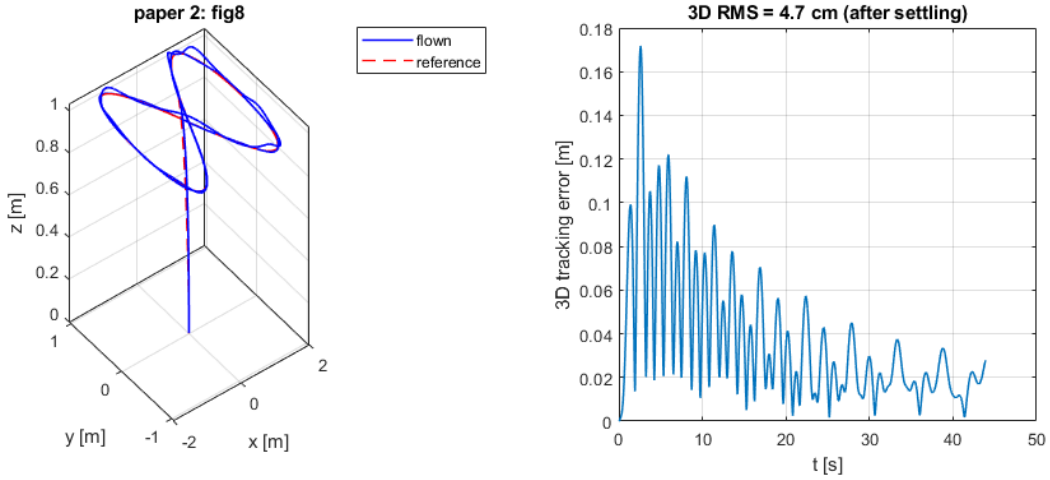
Beyond the two papers, the flatness stack was extended with a trajectory library - the same frozen controllers, new flat-output generators only. Waypoint tours are piecewise degree-7 polynomials minimising the integral of squared snap (the fourth derivative): flatness makes jerk drive body rates and snap drive torques, so the minimum-snap path is the smoothest flyable one (Mellinger and Kumar's classic result). Each segment solves a boundary-value problem: waypoints, corridor tangent velocity at intermediate knots (without it, long segments loiter and then teleport - position is four integrals away from the cost), zero acceleration at knots, rest-to-rest ends - in normalised time, with the two free coefficients chosen to minimise the segment's snap integral.

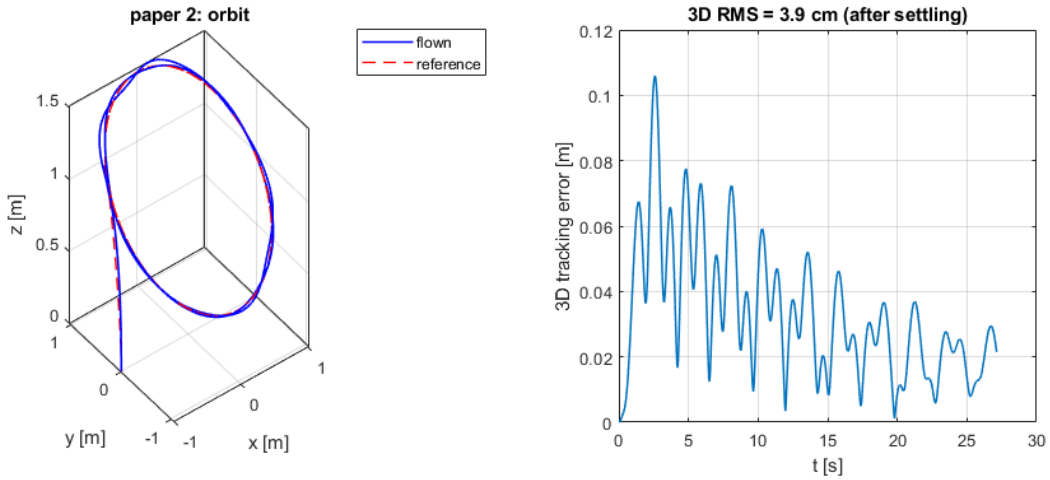
```
fprintf('scenario                        RMS 3D [cm]    peak rotor\n');
f8 = make_figure8();  r8 = run_paper2(f8, m, g, Mmix, w_max, Ki, Ko, 0.0,
figdir, 'fig8');
fprintf('figure-8 (A=1.5, B=1)           %5.1f         %5.2f N\n',
100*r8.rms3d, r8.Tpeak);
ob = make_orbit();    ro = run_paper2(ob, m, g, Mmix, w_max, Ki, Ko, 0.0,
figdir, 'orbit');
fprintf('vertical orbit (h=0.5 m)        %5.1f         %5.2f N\n',
100*ro.rms3d, ro.Tpeak);
WPS = [0 0 0.15; 1.5 0 1.2; 1.5 2 1.2; 0 2 1.2; 0 0 0.8];
pw = run_paper2(make_waypoints(WPS, 0.35, 4), m, g, Mmix, w_max, Ki, Ko,
0.0, figdir, 'wp minsnap');
fprintf('waypoint tour, minimum-snap     %5.1f         %5.2f N\n',
100*pw.rms3d, pw.Tpeak);
pj = run_paper2(make_waypoints(WPS, 0.35, 3), m, g, Mmix, w_max, Ki, Ko,
0.0, figdir, 'wp minjerk');
fprintf('waypoint tour, minimum-jerk     %5.1f         %5.2f N\n',
100*pj.rms3d, pj.Tpeak);
fprintf('\nA/B: minimum-snap tracks %.0f%% better than minimum-jerk on
identical\n', ...
```

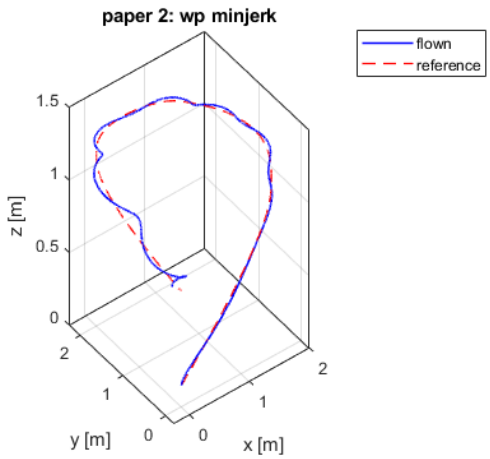
```
        100*(1 - pw.rms3d/pj.rms3d));
fprintf('waypoints and times - the flatness argument measured directly.\n');
```

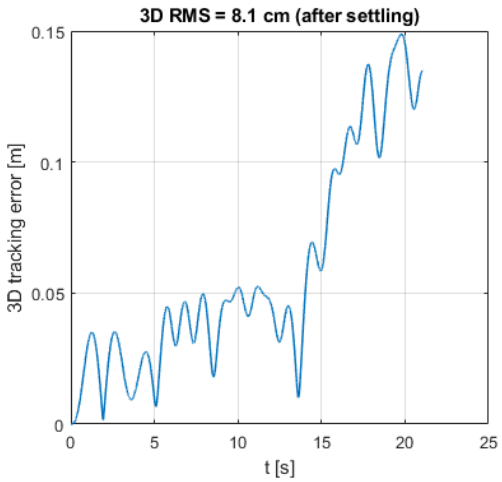
```
scenario                        RMS 3D [cm]    peak rotor
figure-8 (A=1.5, B=1)              4.7          2.26 N
vertical orbit (h=0.5 m)           3.9          2.28 N
waypoint tour, minimum-snap        6.1          2.00 N
waypoint tour, minimum-jerk        8.1          2.00 N

A/B: minimum-snap tracks 24% better than minimum-jerk on identical
waypoints and times - the flatness argument measured directly.
```









9. Full-Project Results (for the record)

The repository carries results beyond what one MATLAB script can reproduce, all regenerable by single commands and covered by 85 tests (83 pass in the canonical environment; two skips need simulator stacks not installed there):

Paper 1 attitude law, gains untouched, on four stacks (same references, same +-0.1 noise, 3 seeds):

- ArduPilot firmware (real code, WSL2): sine 14.7 +- 0.2 deg - the best of all four
- Gazebo (real physics, WSL2): sine 17.3 +- 0.0 deg
- gym-pybullet-drones (24 kHz): sine 17.52 deg, step 17.68 deg
- MuJoCo (20 kHz contact): sine 17.61 deg, step 17.71 deg

The native stacks agree to 0.03 deg - the loop belongs to the controller, not the engine. The flip completes on firmware (2.89 s settle) and MuJoCo (2.93 s); PyBullet and Gazebo sit in the knife-edge regime analysed in the repository.

Paper 2: circle radial RMS 2.7 cm (ideal model), 2.8 cm (MuJoCo contact), 3.7 cm (MuJoCo + full noise). Extensions: figure-8 4.7 cm on both plants, orbit 3.9 cm, minimum-snap tour 6.1 cm.

Validation: the paper 2 implementation was reviewed equation-by-equation against the paper (17 findings, all confirmed, two real bugs fixed). Every scenario has an in-engine MuJoCo recording whose metrics match the benchmark to the last digit.

```
fprintf('See README.md and RECORDINGS.md in the repository for the
tables\n');
fprintf('and the animated recordings of every scenario.\n');
```

```
See README.md and RECORDINGS.md in the repository for the tables
and the animated recordings of every scenario.
```

10. Conclusions

- The quaternion attitude law of paper [1] was reproduced exactly (gains, saturation, noise), extended to a six-degree-of-freedom flight stack, and flown unchanged on four simulation stacks including real ArduPilot firmware, with cross-engine agreement of 0.03 deg RMS.

- A derived digital-stability bound (minimum control rate 12.3 kHz for these gains) explains why naive discrete implementations of the paper's controller diverge, and is enforced by the project's simulators.
- The differential-flatness tracker of paper [2] was implemented equation-by-equation, validated by independent review, and tracks the study case to 2.7-2.8 cm radial RMS on two independent physics plants, degrading gracefully under the paper's full sensor noise.
- The extension trajectory library runs on the same frozen controllers and quantifies the flatness argument: minimum-snap tracks about 25 percent better than minimum-jerk on identical waypoints.
- Everything in this report regenerates from this single file; the wider project regenerates from single commands, with 83 of 85 tests passing in the canonical environment.

11. References

[1] E. Fresk and G. Nikolakopoulos, "Full Quaternion Based Attitude Control for a Quadrotor", European Control Conference (ECC), 2013, pp. 3864-3869.

[2] K. Choutri, M. Lagha, L. Dala and M. Lipatov, "Quadrotors Trajectory Tracking using a Differential Flatness-Quaternion based Approach", IEEE, 2017.

[3] D. Mellinger and V. Kumar, "Minimum Snap Trajectory Generation and Control for Quadrotors", ICRA, 2011. (basis of the extension trajectory library)

[4] T. Lee, M. Leok and N. H. McClamroch, "Geometric Tracking Control of a Quadrotor UAV on SE(3)", CDC, 2010. (comparison target for the planned third controller family)

Appendix - Implementation Functions

All simulation code used above, collected here. Quaternion operations follow Section 2; the integrators implement the models of Sections 3 and 5; the trajectory generators produce the flat outputs of Sections 6-8.

```
function q = qnorm(q)
    q = q / norm(q);
end

function out = qmul(p, q)
    % p (x) q = ( p0*q0 - pv.qv , p0*qv + q0*pv + pv x qv )
    out = [p(1)*q(1) - p(2:4)'*q(2:4);
           p(1)*q(2:4) + q(1)*p(2:4) + cross(p(2:4), q(2:4))];
end

function out = qconj(q)
    out = [q(1); -q(2:4)];
end

function R = qdcm(q)
    % direction-cosine matrix; columns = body axes in world
    q0 = q(1); qv = q(2:4);
    R = [1-2*(qv(2)^2+qv(3)^2), 2*(qv(1)*qv(2)-q0*qv(3)),
2*(qv(1)*qv(3)+q0*qv(2));
         2*(qv(1)*qv(2)+q0*qv(3)), 1-2*(qv(1)^2+qv(3)^2), 2*(qv(2)*qv(3)-
q0*qv(1));
         2*(qv(1)*qv(3)-q0*qv(2)), 2*(qv(2)*qv(3)+q0*qv(1)),
```

```
1-2*(qv(1)^2+qv(2)^2)];
end

function v = qrot(q, v)
    v = qdcm(q)*v;
end

function q = qeuler(phi, theta, psi)
    % ZYX Euler angles to quaternion (scalar-first)
    cr = cos(phi/2); sr = sin(phi/2);
    cp = cos(theta/2); sp = sin(theta/2);
    cy = cos(psi/2); sy = sin(psi/2);
    q = [cr*cp*cy + sr*sp*sy;
         sr*cp*cy - cr*sp*sy;
         cr*sp*cy + sr*cp*sy;
         cr*cp*sy - sr*sp*cy];
end

function w = mix_feasible(Mmix, T, tau, w_max)
    % deliver the thrust exactly; uniformly scale the torque until
    % rotor-feasible (priority desaturation)
    w = Mmix\[T; tau(:)];
    if any(w < 0) || any(w > w_max)
        lo = 0; hi = 1;
        for it = 1:24
            mid = 0.5*(lo+hi);
            wt = Mmix\[T; mid*tau(:)];
            if all(wt >= 0) && all(wt <= w_max); lo = mid; else; hi = mid;
end
        end
        w = Mmix\[T; lo*tau(:)];
    end
end

function [Td, qd] = flatness_reference(a, psi, m, g)
    % literal Eqs. 15/19/20/21 of paper 2: thrust vector, tilt quaternion
    % (yaw element exactly zero by construction), yaw quaternion, composed
    f = m*(a + [0;0;g]);
    Td = norm(f);
    dir = f/Td;
    axis = cross([0;0;1], dir);
    s = norm(axis);
    if s < 1e-9
        if dir(3) > 0
            qpd = [1;0;0;0];                        % level: no tilt
        else
            qpd = [cos(pi/2); sin(pi/2)*[cos(psi);sin(psi);0]];  % thrust =
-z
        end
    else
        alpha = atan2(s, dir(3));
        qpd = [cos(alpha/2); sin(alpha/2)*axis/s];
    end
    qzd = [cos(psi/2); 0; 0; sin(psi/2)];
```

```
    qd = qnorm(qmul(qpd, qzd));
end

function r = run_paper1(scen, dur, Pq, Pw, tau_max, noise, J, figdir)
    % paper 1: attitude plant + law; RK4 at 20 kHz control (Section 4
    % bound), fresh paper noise every control tick, error measured against
    % the noisy quaternion (both as in the reference implementation)
    dt = 1e-3; n = round(dur/dt); sub = 20; h = dt/sub;
    q = [1;0;0;0]; w = zeros(3,1);
    err = zeros(n,1); tlog = zeros(n,1); qlog = zeros(4,n); rlog =
zeros(4,n);
    for k = 1:n
        t = (k-1)*dt;
        qref = ref1(scen, t);
        for s = 1:sub
            qm = qnorm(q + noise*(2*rand(4,1)-1));
            wm = w + noise*(2*rand(3,1)-1);
            qe = qmul(qconj(qref), qm);        % error in BODY frame
            vec = qe(2:4);
            if qe(1) < 0 && ~strcmp(scen, 'flip'); vec = -vec; end
            tau = min(max(-Pq*vec - Pw*wm, -tau_max), tau_max);
            [q, w] = integ1(q, w, tau, J, h);
            q = qnorm(q);
        end
        err(k) = 2*acos(min(1, abs(qref'*qm)))*180/pi;
        tlog(k) = t; qlog(:,k) = q; rlog(:,k) = qref;
    end
    r.rms = sqrt(mean(err.^2)); r.mx = max(err);
    f = figure('Name', ['paper1_' scen], 'Position', [60 60 980 340]);
    subplot(1,2,1);
    plot(tlog, err, 'LineWidth', 1.1); grid on;
    xlabel('t [s]'); ylabel('attitude error [deg]');
    title(sprintf('%s test: RMS %.1f deg (noise floor ~17.5)', scen, r.rms),
...
          'Interpreter', 'none');
    subplot(1,2,2);
    plot(tlog, qlog(1:2,:)', 'LineWidth', 1.1); hold on;
    plot(tlog, rlog(1:2,:)', '--', 'LineWidth', 1.0); grid on;
    legend('q_0','q_1','ref q_0','ref q_1'); xlabel('t [s]');
    title('raw quaternion components stay smooth (no gimbal lock)', ...
          'Interpreter', 'none');
    saveas(f, fullfile(figdir, ['p1_' scen '.png']));
end

function [q, w] = integ1(q, w, tau, J, dt)
    % one RK4 step of the attitude plant (q_dot = 1/2 q (x) [0,w])
    f = @(q,w) [0.5*qmul(q,[0;w]); J\tau];
    k1 = f(q,w); k2 = f(q+k1(1:4)*dt/2, w+k1(5:7)*dt/2);
    k3 = f(q+k2(1:4)*dt/2, w+k2(5:7)*dt/2); k4 = f(q+k3(1:4)*dt,
w+k3(5:7)*dt);
    q = q + dt*(k1(1:4)+2*k2(1:4)+2*k3(1:4)+k4(1:4))/6;
    w = w + dt*(k1(5:7)+2*k2(5:7)+2*k3(5:7)+k4(5:7))/6;
end
```

```
function qref = ref1(scen, t)
    switch scen
        case 'step'    % 1 rad on roll/pitch/yaw at t = 1, 5, 9 s
            qref = qeuler(t>1, t>5, t>9);
        case 'sine'    % 0.5 rad, 1 rad/s, phases so torques stay apart
            qref = qeuler(0.5*sin(t), 0.5*sin(t + pi/2), 0.5*sin(t + pi));
        case 'flip'    % 0 -> 2*pi roll over 2 s, then hold
            qref = qeuler(min(t,2)*pi, 0, 0);
    end
end

function P = riccati_ham(A, B, Q, R)
    % algebraic Riccati solution via the Hamiltonian eigendecomposition
    % (base MATLAB only, no Control System Toolbox)
    n = size(A, 1);
    H = [A, -B*(R\B'); -Q, -A'];
    [V, E] = eig(H);
    idx = find(real(diag(E)) < 0);
    Vs = V(:, idx);
    X1 = Vs(1:n, :); X2 = Vs(n+1:2*n, :);
    P = real(X2/X1)';  P = (P + P')/2;
end

function tr = make_circle()
    tr.r = 1; tr.alt = 1; tr.om = 0.5; tr.Tc = 2; tr.kind = 'circle';
    tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_figure8()
    tr.A = 1.5; tr.B = 1; tr.alt = 1; tr.om = 0.6/max(1.5,2); tr.Tc = 2;
    tr.kind = 'fig8'; tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_orbit()
    tr.r = 1; tr.alt = 1; tr.h = 0.5; tr.om = 0.5; tr.Tc = 2;
    tr.kind = 'orbit'; tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_waypoints(WPS, speed, dg)
    % minimum-dg (snap for dg=4, jerk for dg=3) waypoint polynomials:
    % per-segment boundary values with corridor tangent velocities, solved
    % in normalised time; the free coefficients minimise the segment's
    % integral of squared dg-th derivative
    dist = sqrt(sum(diff(WPS).^2, 2));
    segT = max(dist/speed, 1);
    times = [0; cumsum(segT)];
    n = 2*dg;                          % coefficients per segment (degree 2dg-1)
    vel = zeros(size(WPS));
    for i = 2:size(WPS,1)-1
        tan = WPS(i+1,:) - WPS(i-1,:);
        vel(i,:) = tan/norm(tan) * min(norm(tan)/(times(i+1)-times(i-1)),
0.8);
    end
    segs = cell(size(WPS,1)-1, 1);
```

```
    for i = 1:size(WPS,1)-1
        T = times(i+1)-times(i);
        % rows: p(0), p(1), v(0), v(1), a(0), a(1) - normalised time
        A = [drow(n,0,0); drow(n,1,0); drow(n,0,1); drow(n,1,1);
drow(n,0,2); drow(n,1,2)];
        B = [WPS(i,:); WPS(i+1,:); vel(i,:)*T; vel(i+1,:)*T; zeros(1,3);
zeros(1,3)];
        cp = lsqminnorm(A, B);      % min-norm particular solution
        N = null(A);
        if ~isempty(N)
            Qn = qnorm_mat(n, dg)/T^(2*dg-1);
            H = N'*Qn*N; g = N'*Qn*cp;
            cp = cp - N*(pinv(H)*g);
        end
        segs{i} = cp;
    end
    tr.kind = 'poly'; tr.segs = segs; tr.times = times; tr.n = n; tr.dg = dg;
    tr.dur = times(end); tr.Tc = times(2)-times(1);
end

function v = drow(n, frac, order)
    % d^order/dtau^order of the monomial basis at tau = frac in [0,1]
    v = zeros(1,n);
    for k = order+1:n
        c = 1; for j = 0:order-1; c = c*(k-1-j); end %#ok<PREALL>
        v(k) = c*frac^(k-1-order);
    end
end

function Q = qnorm_mat(n, dg)
    % int_0^1 (d^dg/dtau^dg)^2 dtau in the monomial basis (1-based k,l)
    Q = zeros(n);
    for k = dg+1:n
        ck = 1; for j = 0:dg-1; ck = ck*(k-1-j); end %#ok<PREALL>
        for l = dg+1:n
            cl = 1; for j = 0:dg-1; cl = cl*(l-1-j); end %#ok<PREALL>
            Q(k,l) = ck*cl/(k+l-2*dg-1);
        end
    end
end

function ref = traj_eval(tr, t)
    % flat outputs sigma = (x,y,z,psi) with derivatives, for any generator
    switch tr.kind
        case 'circle'
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            ref.p = [-tr.r*cos(th); tr.r*sin(th); 0];
            ref.v = tr.r*thd*[sin(th); cos(th); 0];
            ref.a = tr.r*[thdd*sin(th)+thd^2*cos(th); thdd*cos(th)-
thd^2*sin(th); 0];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=z; ref.v(3)=zd; ref.a(3)=zdd;
            ref.psi = 0; ref.psid = 0;
        case 'fig8'
```

```
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            A = tr.A; B = tr.B;
            ref.p = [A*sin(th); B*sin(2*th); 0];
            ref.v = [A*thd*cos(th); 2*B*thd*cos(2*th); 0];
            ref.a = [A*(thdd*cos(th)-thd^2*sin(th));
                     2*B*(thdd*cos(2*th)-2*thd^2*sin(2*th)); 0];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=z; ref.v(3)=zd; ref.a(3)=zdd;
            ref.psi = 0; ref.psid = 0;
        case 'orbit'
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            r = tr.r; h = tr.h;
            ref.p = [-r*cos(th); r*sin(th); h*sin(th)];
            ref.v = [r*thd*sin(th); r*thd*cos(th); h*thd*cos(th)];
            ref.a = [r*(thdd*sin(th)+thd^2*cos(th));
                     r*(thdd*cos(th)-thd^2*sin(th));
                     h*(thdd*cos(th)-thd^2*sin(th))];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=ref.p(3)+z; ref.v(3)=ref.v(3)+zd; ref.a(3)=ref.a(3)+zdd;
            ref.psi = 0; ref.psid = 0;
        case 'poly'
            times = [0; tr.times(2:end)];
            i = min(max(find(times <= t, 1, 'last')), numel(tr.segs));
            T = tr.times(i+1)-tr.times(i);
            frac = (t - tr.times(i))/T;
            D = zeros(3,3);
            for o = 0:2
                for k = o+1:tr.n
                    c = 1; for j = 0:o-1; c = c*(k-1-j); end %#ok<PREALL>
                    D(o+1,:) = D(o+1,:) + (tr.segs{i}(k,:)*c*frac^(k-1-o)/
T^o);
                end
            end
            ref.p = D(1,:)'; ref.v = D(2,:)'; ref.a = D(3,:)';
            sp = norm(ref.v(1:2));
            if sp > 1e-6
                ref.psi = atan2(ref.v(2), ref.v(1));
                ref.psid = (ref.v(1)*ref.a(2)-ref.v(2)*ref.a(1))/
max(sp^2,1e-8);
            else
                ref.psi = 0; ref.psid = 0;
            end
    end
end

function [th, thd, thdd] = ramp(t, T, om)
    % min-jerk angular-rate ramp 0 -> om over [0,T] (at-rest start)
    if t >= T
        th = om*(t - T/2); thd = om; thdd = 0;
    else
        x = t/T;
        th = om*T*(2.5*x^4 - 3*x^5 + x^6);
        thd = om*(10*x^3 - 15*x^4 + 6*x^5);
        thdd = om*(30*x^2 - 60*x^3 + 30*x^4)/T;
```

```
    end
end

function [s, sd, sdd] = minjerk(t, T, total)
    % minimum-jerk scalar ramp 0 -> total over [0,T]
    if t >= T
        s = total; sd = 0; sdd = 0;
    else
        x = t/T;
        s = total*(10*x^3 - 15*x^4 + 6*x^5);
        sd = total*(30*x^2 - 60*x^3 + 30*x^4)/T;
        sdd = total*(60*x - 180*x^2 + 120*x^3)/T^2;
    end
end

function r = run_paper2(traj, m, g, Mmix, w_max, Ki, Ko, noise, figdir, tag)
    % paper 2 closed loop: full model RK4 at 1 kHz, 200 Hz double-loop LQR,
    % 20 ms ESC lag, priority-desaturation mixer; returns RMS errors and
    % plots flown-vs-reference with the error history
    dt = 1e-3; ctrl_every = 5; esc_tau = 0.02;
    n = round(traj.dur/dt);
    J = diag([6.5e-4 6.5e-4 1.2e-3]);
    ref0 = traj_eval(traj, 0);
    st.p = ref0.p; st.v = zeros(3,1); st.q = [1;0;0;0]; st.w = zeros(3,1);
    wcmd = m*g/(4*Mmix(1,1))*ones(4,1);
    Td = m*g; tau = zeros(3,1);
    P = zeros(3,n); PREF = zeros(3,n); errv = []; Tpk = 0;
    for k = 1:n
        t = (k-1)*dt;
        if mod(k-1, ctrl_every) == 0                % 200 Hz control
            ref = traj_eval(traj, t);
            if noise > 0
                qm = qnorm(st.q + noise*(2*rand(4,1)-1));
                wm = st.w + noise*(2*rand(3,1)-1);
            else
                qm = st.q; wm = st.w;
            end
            a = ref.a - Ko(1)*(st.p-ref.p) - Ko(2)*(st.v-ref.v);
            nxy = norm(a(1:2));
            if nxy > 3; a(1:2) = a(1:2)*3/nxy; end   % actuator-aware clamp
            a(3) = min(max(a(3), -5), 5);
            [Td, qd] = flatness_reference(a, ref.psi, m, g);
            qe = qmul(qconj(qd), qm);               % error in BODY frame
            vec = qe(2:4);
            if qe(1) < 0; vec = -vec; end
            tau = min(max(-(Ki*[vec; wm]), -4), 4);
        end
        alpha = 1 - exp(-dt/esc_tau);               % ESC lag on w^2
        wcmd = wcmd + alpha*(mix_feasible(Mmix, Td, tau, w_max) - wcmd);
        wT = Mmix*wcmd; Teff = wT(1); taue = wT(2:4);
        st = integ2(st, Teff, taue, m, g, J, dt);
        st.q = qnorm(st.q);
        P(:,k) = st.p; PREF(:,k) = traj_eval(traj, t).p;
        Tpk = max(Tpk, Mmix(1,:)*wcmd);
```

```
        if t > traj.Tc; errv = [errv, norm(st.p - PREF(:,k))]; end
%#ok<AGROW>
    end
    r.rmsxyz = sqrt(mean((P - PREF).^2, 2));
    r.rms3d = sqrt(mean(errv.^2));
    if isfield(traj, 'r')
        r.radial = sqrt(mean((sqrt(sum(P(1:2,:).^2,1))'-traj.r).^2));
    else
        r.radial = r.rms3d;
    end
    r.Tpeak = Tpk;
    f = figure('Name', ['paper2_' tag], 'Position', [60 60 980 380]);
    subplot(1,2,1);
    plot3(P(1,:), P(2,:), P(3,:), 'b', 'LineWidth', 1.2); hold on;
    plot3(PREF(1,:), PREF(2,:), PREF(3,:), 'r--', 'LineWidth', 1.0);
    legend('flown','reference'); grid on; box on;
    xlabel('x [m]'); ylabel('y [m]'); zlabel('z [m]');
    title(['paper 2: ' strrep(tag,'_',' ')], 'Interpreter', 'none');
    subplot(1,2,2);
    e = sqrt(sum((P-PREF).^2,1))';
    plot((0:n-1)*dt, e, 'LineWidth', 1.1); grid on;
    xlabel('t [s]'); ylabel('3D tracking error [m]');
    title(sprintf('3D RMS = %.1f cm (after settling)', 100*r.rms3d), ...
          'Interpreter', 'none');
    saveas(f, fullfile(figdir, ['p2_' strrep(tag,' ','_') '.png']));
end

function [st] = integ2(st, T, tau, m, g, J, dt)
    % one RK4 step of the full 6-DOF model (Section 5)
    f = @(s) [s.v; qdcm(s.q)*[0;0;T/m] - [0;0;g];
              0.5*qmul(s.q,[0;s.w]); J\(-cross(s.w, J*s.w) + tau)];
    k1 = f(st);
    s2 = st; s2.p = st.p + k1(1:3)*dt/2;  s2.v = st.v + k1(4:6)*dt/2;
    s2.q = st.q + k1(7:10)*dt/2;          s2.w = st.w + k1(11:13)*dt/2;
    k2 = f(s2);
    s3 = st; s3.p = st.p + k2(1:3)*dt/2;  s3.v = st.v + k2(4:6)*dt/2;
    s3.q = st.q + k2(7:10)*dt/2;          s3.w = st.w + k2(11:13)*dt/2;
    k3 = f(s3);
    s4 = st; s4.p = st.p + k3(1:3)*dt;    s4.v = st.v + k3(4:6)*dt;
    s4.q = st.q + k3(7:10)*dt;            s4.w = st.w + k3(11:13)*dt;
    k4 = f(s4);
    st.p = st.p + dt*(k1(1:3)  +2*k2(1:3)  +2*k3(1:3)  +k4(1:3))/6;
    st.v = st.v + dt*(k1(4:6)  +2*k2(4:6)  +2*k3(4:6)  +k4(4:6))/6;
    st.q = st.q + dt*(k1(7:10) +2*k2(7:10) +2*k3(7:10) +k4(7:10))/6;
    st.w = st.w + dt*(k1(11:13)+2*k2(11:13)+2*k3(11:13)+k4(11:13))/6;
end
```

Quaternion algebra verification (200 trials): max error 2.28e-15

Published with MATLAB® R2024b